09. 메시징 게이트웨이

이 장에서는 Hermes Agent를 메시징 플랫폼과 연동하여, 터미널 밖에서도 에이전트와 대화하는 방법을 다룹니다. 터미널 앞에 앉아 있지 않아도 스마트폰의 Telegram이나 Discord에서 에이전트에게 작업을 지시하고 결과를 받을 수 있습니다. 크론 작업의 알림을 받거나, 외출 중에 서버 상태를 확인하거나, 팀원들과 하나의 에이전트를 공유하는 것도 가능합니다.

Hermes Agent는 Telegram, Discord, WhatsApp, Slack, Signal 등 **22개 플랫폼**을 지원합니다. 명령 하나로 설정된 모든 플랫폼에 동시에 연결할 수 있습니다.

이 장에서 다루는 내용

- 지원 플랫폼 개요 (세션 관리, DM 페어링, PII 보호)
- 텔레그램 연동 (음성, 스트리밍 응답)
- 디스코드 연동 (스레드 기반 세션, 팀 활용)
- Microsoft Teams 연동 (Graph API + 웹훅, v0.14 정식 통합)

09-1. 지원 플랫폼 개요

- CLI와 게이트웨이의 차이
- 지원 플랫폼
- 게이트웨이 시작
  - 대화형 설정 마법사
  - 포그라운드 실행
  - 서비스로 설치 (권장)
- 세션 관리
  - 플랫폼 간 공유와 분리
  - 세션 리셋 모드
- 보안
  - DM 페어링
  - PII 보호 (개인정보 해싱)
- 게이트웨이 공통 기능
  - 멀티 이미지 발송
  - 승인 버튼 (Slack/Telegram)
  - 웹훅 직접 전달 모드 (Zero-LLM Push)
  - 통합 프락시 지원
- 음성 메시지 지원
  - 중앙화된 오디오 라우팅
  - STT/TTS 설정
  - 음성 메시지 처리 흐름
- 플랫폼별 추가 옵션
  - Slack — strict mention과 채널-스킬 바인딩
- 플러그인 호스트로 확장
  - 번들 플러그인 자동 로드
- 플랫폼 선택 가이드
- 게이트웨이 전용 명령어
- 매체 전달 — 이미지·문서·음성
- 버전별 주요 변경

Hermes Agent는 **메시징 게이트웨이**를 통해 여러 메시징 플랫폼과 연결됩니다. 모든 플랫폼 연결은 게이트웨이 프로세스 하나가 관리합니다.

CLI와 게이트웨이의 차이

`hermes` (CLI)는 터미널에서 직접 대화하는 모드입니다. 터미널을 닫으면 끝납니다. 반면 **게이트웨이**는 Telegram, Discord 같은 외부 플랫폼의 메시지를 받아 에이전트에 전달하는 **상시 실행 프로세스**입니다. 크론 작업의 결과를 메시지로 전달하는 것도 게이트웨이의 역할입니다.

```
CLI 모드:      사용자 ↔ 터미널 ↔ 에이전트
게이트웨이 모드: 사용자 ↔ Telegram/Discord/… ↔ 게이트웨이 프로세스 ↔ 에이전트
```

지원 플랫폼

v0.14.0 기준 **22개 플랫폼**을 사용할 수 있습니다.

플랫폼	연동 방식	비고
Telegram	봇 API	가장 쉬운 설정, 음성 메시지 지원
Discord	봇 토큰	팀 사용에 적합, 스레드 기반 세션
WhatsApp	Baileys (비공식)	PII 해싱 적용
Slack	앱 토큰	업무 환경 연동, 멀티 워크스페이스
Signal	Signal CLI	마크다운→bodyRanges, 답글·이모지 반응, 멀티 이미지, PII 해싱
Mattermost	API	자체 호스팅 환경에 적합
Matrix	SDK	분산형 메시징 프로토콜, E2EE 지원
HomeAssistant	API	스마트홈 연동
Email	IMAP/SMTP	이메일 기반 상호작용
SMS (Twilio)	Twilio API	웹훅 시그니처 검증
DingTalk	봇 API	중국 기업용 메신저
WeCom (WebSocket)	WebSocket	기업 위챗(Enterprise WeChat) 표준
WeCom (Callback)	웹훅 콜백	자체 빌드 엔터프라이즈 앱
WeChat / 微信	iLink Bot API	개인 계정 연동
Feishu/Lark	WebSocket/Webhook	바이트댄스 기업 메신저
iMessage	BlueBubbles 브리지	Apple 메시징 생태계
QQBot	공식 API v2	QR 스캔 설정, 스트리밍 커서, DM/그룹 정책
Tencent 元宝 (Yuanbao)	네이티브 어댑터	텍스트·미디어
Microsoft Teams	Graph API + 웹훅	인증·웹훅·파이프라인·송신 end-to-end
LINE	LINE Messaging API	일본·한국·대만 사용자 비중이 큰 플랫폼

플랫폼	연동 방식	비고
SimpleX Chat	SimpleX 프로토콜	사용자 ID 없는 분산형 프라이버시 메신저
API Server	REST API	커스텀 클라이언트 연동

비고 칸은 플랫폼 고유 특성만 표시합니다. 버전별 도입 시점은「버전별 주요 변경」절을 따로 참고합니다.

게이트웨이 시작

대화형 설정 마법사

처음 설정할 때는 대화형 마법사를 사용합니다.

```
hermes gateway setup
```

플랫폼 선택, 토큰 입력, 기본 설정을 단계별로 안내합니다.

포그라운드 실행

```
hermes gateway run
```

터미널에서 직접 실행하며 로그를 바로 확인할 수 있습니다. 테스트나 디버깅에 적합합니다. 터미널을 닫으면 게이트웨이도 종료됩니다.

서비스로 설치 (권장)

게이트웨이를 상시 실행하려면 OS 서비스로 등록합니다.

```
hermes gateway install   # 서비스 등록
hermes gateway start     # 서비스 시작
```

Linux에서는 systemd 유저 서비스(`~/.config/systemd/user/hermes-gateway.service`)로, macOS에서는 launchd(`~/Library/LaunchAgents/ai.hermes.gateway.plist`)로 설치됩니다. 부팅 시 자동 시작되고, 비정상 종료 시 자동 재시작됩니다.

서비스 관리 명령은 다음과 같습니다.

명령어	설명
`hermes gateway install`	OS 서비스로 등록
`hermes gateway uninstall`	서비스 제거
`hermes gateway start`	서비스 시작
`hermes gateway stop`	서비스 중지
`hermes gateway restart`	서비스 재시작
`hermes gateway status`	상태 확인
`hermes gateway run`	포그라운드 실행 (서비스 미사용)

로그 확인

서비스로 실행 중일 때 로그를 보려면 Linux에서는 `journalctl --user -u hermes-gateway -f`, macOS에서는 `~/.hermes/logs/gateway.log`를 확인합니다.

세션 관리

플랫폼 간 공유와 분리

대화 세션은 플랫폼별로 **완전히 분리**됩니다. Telegram에서 나눈 대화를 Discord에서 이어가는 것은 불가능합니다. 세션 키가 `agent:main:<플랫폼>:<채팅ID>` 형식이므로, 같은 사람이라도 플랫폼이 다르면 별도 세션입니다.

다만 **메모리 파일**(MEMORY.md, USER.md)은 모든 플랫폼과 CLI가 공유합니다. 예를 들어 Telegram에서 "나는 Python을 선호해"라고 말해서 메모리에 저장되면, Discord에서도 에이전트가 그 사실을 알고 있습니다. 공유되는 것은 대화 기록이 아니라 에이전트가 학습한 **사실과 선호**입니다.

세션 리셋 모드

세션 리셋 모드는 `session_reset_mode` 설정으로 제어합니다. 네 가지 모드가 있습니다.

모드	작동
`daily`	매일 오전 4시에 리셋
`idle`	유휴 시간 초과 시 리셋
`both` (기본값)	daily + idle 모두 적용
`none`	자동 리셋 없음

유휴 타임아웃(`session_idle_timeout`)의 기본값은 v0.14 기준 **1440분(24시간)** 입니다. 설정 파일에서 자유롭게 변경할 수 있습니다.

```
# config.yaml 예시
session_reset_mode: both        # daily, idle, both, none
session_idle_timeout: 120       # 분 단위 (예: 2시간)
```

세션 타임아웃 설정

기본값 1440분(24시간)은 개인 사용에 적합합니다. 팀 환경이나 보안이 중요한 경우 120분 등 짧은 값으로 조정하세요.

보안

DM 페어링

미인가 사용자의 DM을 처리하는 방식을 설정합니다.

- **pair**: 페어링 코드를 요구
- **ignore**: 무시

```
unauthorized_dm_behavior: pair
```

페어링 코드 세부 규격은 다음과 같습니다.

항목	값
코드 길이	8자
알파벳	32자 비모호 문자셋 (0/O, 1/I 등 혼동 문자 제외)
코드 만료	1시간
최대 대기 코드 수	3개
요청 속도 제한	10분당 1회
실패 잠금	5회 실패 시 1시간 잠금

PII 보호 (개인정보 해싱)

WhatsApp, Signal, Telegram 플랫폼에서는 사용자 ID를 해싱한 뒤 LLM에 전달합니다. 실제 전화번호나 사용자 식별자가 외부 LLM 제공자에 노출되지 않도록 보호하는 장치입니다. 해당 플랫폼에서 자동으로 적용되며, 별도 설정이 필요 없습니다.

게이트웨이 공통 기능

플랫폼에 관계없이 게이트웨이가 공통으로 제공하는 기능입니다.

멀티 이미지 발송

Telegram·Discord·Slack·Mattermost·Email·Signal에서 한 메시지에 여러 장의 이미지를 묶어 보낼 수 있습니다 (`send_multiple_images`). 도구 호출 한 번으로 갤러리·앨범 형식의 답신이 가능합니다.

승인 버튼 (Slack/Telegram)

위험 명령에 대한 승인 요청이 채팅에 뜰 때, `/approve` · `/deny`를 직접 타이핑하는 대신 **인라인 버튼**으로 처리합니다. Slack에서는 스레드 문맥을 그대로 유지하고, Telegram에서는 이모지 반응으로 승인 상태를 보여 줍니다. 미인가 사용자가 승인 버튼을 눌러도, 인증을 통과해야만 명령이 실행됩니다.

웹훅 직접 전달 모드 (Zero-LLM Push)

웹훅으로 받은 페이로드를 LLM을 거치지 않고 곧바로 채널에 보낼 수 있습니다. 알림, 업타임 체크, 이벤트 스트림처럼 본문이 이미 사람이 읽기 좋은 형식이면 LLM 토큰 비용 없이 그대로 전달할 수 있습니다.

통합 프락시 지원

SOCKS 프락시, `DISCORD_PROXY`, 시스템 프락시 자동 감지를 모든 플랫폼에서 공통으로 지원합니다. 사내 방화벽 뒤에서도 별도 설정 없이 게이트웨이를 연결할 수 있습니다.

음성 메시지 지원

Hermes Agent는 **Telegram, Discord, WhatsApp**에서 음성 메시지를 수신할 수 있습니다. 음성 메시지가 도착하면 STT(Speech-to-Text)로 텍스트 변환 후 에이전트에 전달됩니다. 응답은 TTS(Text-to-Speech)로 음성 메시지로 되돌려 보낼 수도 있고, 텍스트로 응답할 수도 있습니다.

중앙화된 오디오 라우팅

음성 메시지 처리(STT·TTS, 노이즈·게이트·포맷 변환)는 게이트웨이 한 곳에서 처리합니다. FLAC 코덱을 지원하고, Telegram에서 오디오 파일이 메시지 한도를 넘으면 자동으로 **문서 첨부로 전환해 보냅니다**.

STT/TTS 설정

```
# config.yaml
stt:
  provider: openai          # openai, whisper, deepgram
  model: whisper-1

tts:
  provider: openai           # openai, elevenlabs, piper
  model: tts-1
  voice: nova                # alloy, echo, fable, onyx, nova, shimmer
  respond_with_voice: false  # true: 항상 음성 응답, false: 텍스트 응답
```

- `respond_with_voice: true`로 설정하면 에이전트의 응답도 음성 메시지로 전송됩니다.
- STT provider로 로컬 Whisper를 사용하면 외부 API 호출 없이 음성을 처리할 수 있습니다.

음성 메시지 처리 흐름

```
사용자 음성 메시지 → STT 변환 → 에이전트 처리 → 텍스트/TTS 응답
```

플랫폼별 추가 옵션

플랫폼 어댑터별로 추가 설정이 있는 경우를 모았습니다. 기본 연동 절차는 9장의 플랫폼별 절을 참고하세요.

Slack — strict_mention과 채널-스킬 바인딩

```
slack:
  strict_mention: true   # 봇을 명시적으로 멘션했을 때만 응답
  channel_skill_bindings:
    "C012ABC...":  ["incident-response", "oncall-runbook"]
    "C098XYZ...":  ["sales-research"]
```

`strict_mention: true`로 두면 같은 채널에서 봇이 한 번 끼어든 뒤 후속 메시지에 자동으로 응답하는 일이 차단됩니다. `channel_skill_bindings`는 채널별로 활성화할 스킬 목록을 묶는 기능으로, 인시던트 채널에는 운영 스킬만, 영업 채널에는 리서치 스킬만 노출하는 식의 구성을 할 수 있습니다.

모든 게이트웨이 명령(`/approve`, `/sethome`, `/queue` 등)은 **Slack 네이티브 슬래시 명령**으로 등록됩니다. Discord·Telegram과 자동완성이 같은 수준입니다.

플러그인 호스트로 확장

v0.12부터 게이트웨이는 **플러그인 호스트**로 작동합니다. 새 플랫폼 어댑터는 코어 코드를 고치지 않고 플러그인처럼 바로 추가할 수 있습니다. Microsoft Teams가 이 방식을 처음 적용한 사례입니다.

번들 플러그인 자동 로드

Spotify, Google Meet, Langfuse 옵저버빌리티, hermes-achievements가 **번들 플러그인**으로 함께 제공됩니다. 게이트웨이를 시작하면 자동으로 감지되어 도구·페이지·이벤트 훅에서 사용할 수 있습니다. 웹 대시보드의 Plugins 탭에서 각 플러그인을 켜거나 끌 수 있습니다.

플랫폼 선택 가이드

어떤 플랫폼을 사용할지는 용도와 환경에 따라 다릅니다. 자주 비교되는 6개 플랫폼만 추렸습니다.

기준	Telegram	Discord	Slack	WhatsApp	Signal	Email
설정 난이도	쉬움	보통	보통	어려움	어려움	보통
개인 사용	최적	좋음	보통	좋음	좋음	보통
팀 사용	보통	최적	최적	보통	제한적	좋음
음성 메시지	지원	지원	미지원	지원	미지원	미지원
봇 API 안정성	높음	높음	높음	낮음(비공식)	보통	높음
모바일 접근성	높음	높음	높음	높음	높음	높음

추천 조합:

- **개인 사용자** — Telegram 단독으로 충분합니다. 설정이 가장 간단하고 봇 API가 안정적입니다.
- **팀/커뮤니티** — Discord를 메인으로 사용하고, 관리자는 Telegram DM을 보조로 설정합니다.
- **업무 환경** — 조직에서 이미 Slack을 사용 중이라면 Slack 연동이 자연스럽습니다.
- **비기술 사용자** — WhatsApp은 별도 앱 설치 없이 사용할 수 있지만, Baileys 기반의 비공식 연동이라 안정성이 낮습니다.
- **자체 호스팅** — Mattermost나 Matrix를 사용하면 데이터가 외부로 나가지 않습니다.
- **스마트홈** — HomeAssistant 연동으로 음성 비서처럼 사용할 수 있습니다.
- **커스텀 통합** — API Server를 통해 자체 프런트엔드나 서비스에서 Hermes를 호출할 수 있습니다.

여러 플랫폼 동시 사용

`hermes gateway`는 설정된 모든 플랫폼에 동시에 연결됩니다. Telegram과 Discord를 동시에 운영할 수 있습니다. 단, 대화 세션은 플랫폼별로 분리되며, 공유되는 것은 `MEMORY.md` · `USER.md`에 저장된 사실·선호 정보입니다.

게이트웨이 전용 명령어

명령어	설명
`/stop`	작업 중단
`/sethome`	크론 출력 전달 채널 설정
`/title [name]`	세션 이름 설정
`/approve [session|always]`	보류 중인 위험 명령 승인
`/deny`	보류 중인 위험 명령 거부
`/queue <prompt>`	다음 턴에 처리할 프롬프트를 큐에 넣음
`/restart`	진행 중 작업 정리 후 게이트웨이 재시작
`/update`	Hermes Agent 업데이트

플랫폼별로 지원되는 명령이 다소 다를 수 있으나 위 명령은 v0.14 기준 Telegram·Discord·Slack 어디서나 작동합니다. Slack은 추가로 네이티브 슬래시 명령으로 등록되어 자동완성이 가능합니다.

매체 전달 — 이미지·문서·음성

게이트웨이는 에이전트 응답 안에 들어 있는 **절대 경로**를 자동으로 찾아 네이티브 첨부로 전달합니다. 예를 들어 응답 본문에 `/home/<user>/.hermes/cache/diagram.png` 이 그대로 적혀 있으면, 게이트웨이가 그 경로를 본문에서 떼어 내고 텔레그램 사진 메시지·디스코드 첨부 형태로 전송합니다. 사용자 입장에서는 "경로 문자열"이 아니라 곧장 보이는 매체로 도착합니다. 스킬 출력에서 매체를 다룰 때 가장 단순한 방법입니다.

이 자동 처리에 두 가지 디렉티브로 세밀한 제어를 더할 수 있습니다.

디렉티브	효과	쓰임새
`[[audio_as_voice]]`	오디오 파일을 **네이티브 음성 메시지 버블**로 승격 (Telegram·WhatsApp 등 지원 플랫폼)	TTS 응답을 운전 중에도 바로 들을 수 있게
`[[as_document]]`	매체를 압축 미리보기 대신 **문서/파일 첨부**로 전달	고해상도 스크린샷·차트 원본 픽셀 보존 (Telegram `sendPhoto` 는 ~200KB·1280px로 재압축)

사용 방법은 응답 본문 어디든(보통 마지막 줄) 디렉티브를 한 번 적어 두는 것입니다. 디렉티브 자체는 전달 전에 본문에서 strip되어 사용자에게는 보이지 않습니다.

```
렌더링한 차트입니다.

/home/<user>/.hermes/cache/chart-q4-2025.png

[[as_document]]
```

작동 단위는 **응답 한 건 전체**입니다. 한 응답에 여러 이미지 경로가 들어 있으면 디렉티브를 한 번 적는 것만으로 모두 같은 방식(문서 첨부 또는 음성 버블)으로 전달됩니다. 응답마다 매체별로 따로 제어하지는 않습니다 — 그래서 적용 범위가 명확합니다.

문서/파일 첨부 경로가 따로 없는 플랫폼(예: SMS)에서는 디렉티브가 의미를 잃고, 그 플랫폼이 가진 첨부 방식으로 폴백합니다.

버전별 주요 변경

- **v0.14.0** — Microsoft Teams 정식 end-to-end(인증·웹훅·파이프라인·송신), LINE, SimpleX Chat 추가 (22개 플랫폼)
- **v0.12.0** — 게이트웨이 플러그인 호스트화 (Microsoft Teams 첫 플러그인 사례), Tencent Yuanbao 어댑터
- **v0.11.0** — QQBot 공식 API v2
- **v0.9.0** — WeCom Callback, WeChat(iLink Bot API), iMessage(BlueBubbles)

09-2. 텔레그램 연동

- 자동 연결 (QR, 권장)
- 수동 연결 (BotFather)
- 설정
- 게이트웨이 시작
- 사용
- 스트리밍 응답
- 크론 출력 전달
- Webhook 모드
- 그룹·포럼 허용 목록과 멘션 게이팅
- 그룹 대화 관찰 모드
- 스킬 기반 슬래시 메뉴
- 음성 메시지
  - STT를 끄고 원본 오디오를 에이전트에 넘기기
  - /voice 명령
  - 음성 활용 예시
- 미디어 처리
- Private Chat Topics
  - 활용 예시
  - 설정
  - 작동 방식
- 실전 예시: 멀티스텝 작업

Telegram은 Hermes Agent와 연동하기 가장 쉬운 플랫폼입니다. 봇을 연결하는 방법은 두 가지입니다. 최근 버전에서는 `hermes setup`이 **자동(QR)**과 **수동(BotFather)** 중 하나를 고르게 합니다.

자동 연결 (QR, 권장)

`hermes setup`의 Telegram 단계에서 자동 방식을 고르면, 화면에 QR 코드가 표시됩니다. 휴대폰의 Telegram으로 QR을 스캔하고 앱에서 확인하면 연결이 끝납니다. 봇 토큰을 직접 복사·붙여넣을 필요가 없습니다. 웹 대시보드에서도 같은 QR 온보딩을 쓸 수 있습니다.

수동 연결 (BotFather)

직접 봇을 만들어 토큰을 넣고 싶다면 BotFather 방식을 씁니다.

1. Telegram에서 **@BotFather**와 대화를 시작합니다
2. `/newbot` 명령으로 새 봇을 생성합니다
3. 봇 이름과 사용자명을 설정합니다
4. 발급된 **봇 토큰**을 복사합니다

설정

`~/.hermes/.env`에 봇 토큰을 추가합니다.

```
TELEGRAM_BOT_TOKEN=1234567890:ABCdefGHIjklMNOpqrSTUvwxyz
```

허용된 사용자를 설정합니다 (선택).

```
# config.yaml
telegram:
  allowed_users:
    - "your_telegram_user_id"
```

게이트웨이 시작

```
hermes gateway
```

Telegram 봇이 온라인으로 표시되면 성공입니다.

사용

Telegram에서 봇에게 메시지를 보내면 Hermes Agent가 응답합니다.

- 일반 텍스트 메시지 → 대화
- `/new` → 새 세션 시작
- `/model` → 모델 전환
- 이미지 전송 → 비전 분석 (보조 모델 필요)

스트리밍 응답

Telegram에서는 **프로그레시브 메시지 편집** 방식으로 스트리밍 응답을 구현합니다. 에이전트가 응답을 생성하는 동안 메시지가 실시간으로 업데이트됩니다.

- 업데이트 간격: 0.3초
- 버퍼 임계값: 40자 (새로 생성된 텍스트가 40자 이상일 때 메시지 편집 발생)

긴 응답이 타이핑되는 것처럼 점진적으로 나타나므로, 에이전트가 작업 중이라는 것을 시각적으로 확인할 수 있습니다.

크론 출력 전달

```
/sethome
```

이 명령을 실행한 채팅방으로 크론 작업의 결과가 전달됩니다.

> 보안 주의
>
> 봇에 터미널 접근 권한이 있다면, `GATEWAY_ALLOW_ALL_USERS=true`로 설정하지 마세요. 반드시 `allowed_users`를 설정하거나 DM 페어링을 사용하세요.

Webhook 모드

기본적으로 Telegram 어댑터는 **롱 폴링**(long polling) 방식으로 메시지를 수신합니다. `TELEGRAM_WEBHOOK_URL` 환경 변수를 설정하면 HTTP Webhook 서버로 전환됩니다. Fly.io, Railway 같은 클라우드 플랫폼은 인바운드 HTTP 트래픽이 들어오면 중지된 머신을 자동으로 깨우므로, Webhook 모드가 적합합니다.

```
# ~/.hermes/.env
TELEGRAM_WEBHOOK_URL=https://my-hermes.fly.dev/webhook
TELEGRAM_WEBHOOK_PORT=8443          # 로컬 리스닝 포트 (기본값: 8443)
TELEGRAM_WEBHOOK_SECRET=my-secret   # 업데이트 검증용 시크릿 토큰 (웹훅 모드 필수)
```

웹훅 모드에서는 **`TELEGRAM_WEBHOOK_SECRET` 이 필수**입니다(누군가 게이트웨이 URL을 추측해 직접 호출하는 시도를 막기 위함). `TELEGRAM_WEBHOOK_URL`을 설정하지 않으면 기존과 동일하게 롱 폴링으로 작동합니다.

그룹·포럼 허용 목록과 멘션 게이팅

그룹 채팅에서 봇이 모든 메시지에 반응하면 불필요한 응답이 발생합니다. Discord 어댑터와 동일한 **멘션 게이팅**을 Telegram 그룹에도 적용할 수 있고, v0.11부터는 **그룹·포럼 단위의 허용 목록**도 같이 제공됩니다.

```
# config.yaml
telegram:
  require_mention: true
  mention_patterns:
    - "(?i)hey\\s+hermes"
    - "(?i)hermes야"
  free_response_chats:
    - "-100123456789"   # 게이팅 우회할 특정 채팅 ID
  allowed_chats:
    - "-100123456789"          # 이 그룹만 허용
    - "-100123456789:42"       # 슈퍼그룹의 특정 포럼 토픽만 허용
```

게이팅이 활성화되면 그룹에서는 슬래시 명령, 봇에 대한 답장, `@봇유저네임` 멘션, `mention_patterns`에 매칭되는 메시지만 처리합니다. `allowed_chats`가 설정돼 있으면 거기에 없는 그룹/포럼은 **아예 무시**됩니다. DM은 제한 없이 그대로 작동합니다.

그룹 대화 관찰 모드

멘션 게이팅을 켜면 봇은 멘션되지 않은 메시지를 완전히 무시합니다. 그런데 나중에 멘션할 때 그때까지의 대화 맥락을 알면 유용한 경우가 있습니다. `observe_unmentioned_group_messages`를 켜면 봇이 그룹 대화를 **세션 트랜스크립트에 기록**하되 **자동 응답은 하지 않습니다**. 이후 `@봇` 멘션이나 봇에 대한 답장이 들어오면 관찰된 맥락을 참고해 응답합니다.

```
# config.yaml
telegram:
  allowed_chats:
    - "-1001234567890"
  group_allowed_chats:
    - "-1001234567890"
  require_mention: true
  observe_unmentioned_group_messages: true
```

```
# 환경변수로도 설정 가능
TELEGRAM_ALLOWED_CHATS=-1001234567890
TELEGRAM_GROUP_ALLOWED_CHATS=-1001234567890
TELEGRAM_OBSERVE_UNMENTIONED_GROUP_MESSAGES=true
```

`allowed_chats`는 봇이 **응답할** 채팅을, `group_allowed_chats`는 **관찰용 공유 세션**을 허용하는 채팅을 지정합니다. 관찰 모드를 쓸 때는 두 목록에 같은 채팅 ID를 넣으면 됩니다. 관찰된 메시지에는 `[닉네임|user_id]` 태그가 붙고, 멘션으로 트리

거된 턴에는 안전 프롬프트가 추가되어 모델이 관찰 내용을 "지시"가 아닌 "맥락"으로 취급합니다.

> ⚠️ ***프라이버시 모드 해제 필요**: 이 기능은 Telegram이 일반 그룹 메시지를 봇에 전달해야 작동합니다. BotFather에서 프라이버시 모드를 비활성화하거나, 봇을 그룹 관리자로 승격해야 합니다(위 "그룹·포럼 허용 목록" 절 참고).*

스킬 기반 슬래시 메뉴

활성화된 스킬이 Telegram 슬래시 명령 메뉴에 자동 등록됩니다. 사용자가 `/` 를 입력하면 사용 가능한 스킬 목록이 자동완성으로 표시됩니다. Telegram Bot API는 최대 100개의 명령을 허용하므로, 초과분은 메뉴에 표시되지 않지만 직접 입력하면 정상 실행됩니다.

음성 메시지

Telegram에서 음성 메시지를 보내면 Hermes Agent가 자동으로 STT(Speech-to-Text)를 통해 텍스트로 변환한 뒤 처리합니다. STT 자동 변환은 **기본적으로 활성화**되어 있으며, 음성 파일은 `.ogg` 형식으로 캐시 디렉터리에 다운로드됩니다.

STT를 끄고 원본 오디오를 에이전트에 넘기기

`config.yaml` 에서 `stt.enabled: false` 로 두면 게이트웨이가 오디오를 받아 캐시 디렉터리에 다운로드만 하고 전사하지 않습니다. 에이전트에는 다음과 같은 마커가 전달되어, 화자 분리(diarization)나 별도 전사 도구처럼 STT 대체 처리를 직접 다룰 수 있습니다.

```
[The user sent a voice message: /home/<user>/.hermes/cache/audio/<hash>.ogg]
```

/voice 명령

`/voice` 명령으로 음성 응답 모드를 제어할 수 있습니다.

모드	작동
`off`	음성 응답 비활성화 (텍스트로만 응답)
`voice_only`	음성 메시지에 대해서만 음성으로 응답
`all`	모든 응답을 음성 메시지로 전송

```
/voice voice_only
```

음성 모드 설정은 `~/.hermes/gateway_voice_mode.json`에 저장되어 게이트웨이를 재시작해도 유지됩니다.

음성 활용 예시

```
사용자: [음성 메시지] "서버 디스크 용량 확인해줘"
Hermes: 현재 디스크 사용량입니다.
        /dev/sda1  50G  32G  16G  67% /
        /dev/sdb1 200G 180G  15G  93% /data
        ⚠️ /data 파티션이 93%입니다. 정리가 필요합니다.
```

`/voice all` 모드에서는 위 응답이 음성 메시지로도 전송됩니다. 운전 중이나 이동 중에 유용합니다.

미디어 처리

Telegram에서 이미지를 전송하면 **비전 보조 모델**(auxiliary vision model)이 이미지를 분석합니다. 캡션을 함께 보내면 해당 지시사항에 맞춰 분석합니다.

게이트웨이에서 텔레그램으로 보낼 때도 한 메시지에 여러 장의 이미지를 묶어 보낼 수 있습니다(`send_multiple_images`, 9-1절 참고). 또 마크다운 표가 본문에 들어 있으면, 텔레그램이 그대로 표를 그릴 수 없으므로 **행 그룹 단위 불릿**으로 변환되어 가독성을 유지합니다. 오디오 파일이 메시지 한도를 넘으면 자동으로 **문서 첨부로 폴백**합니다.

> 💡 ***20MB 한도를 넘어서: 로컬 Bot API 서버**: Telegram 공개 Bot API는 `getFile` 다운로드를 20MB로 제한합니다. 더 긴 음성 메모나 큰 비디오를 받으려면 `telegram-bot-api` 서버를 로컬로 운영하고 `platforms.telegram.extra.base_url`을 그쪽으로 돌리면 한도가 2GB까지 올라갑니다(Hermes가 자동으로 내부 한도를 함께 올림). 절차는 공식 Telegram 메시징 가이드의 "Large Files (>20MB)" 절을 참고하세요.*

지원되는 미디어 유형:

- **이미지** -- 스크린샷 분석, 오류 메시지 해석, 다이어그램 설명
- **파일** -- 텍스트 파일, 로그 파일, CSV 등을 읽고 분석
- **음성** -- STT 변환 후 텍스트로 처리

```
사용자: [스크린샷 전송] "이 에러 메시지 해결 방법 알려줘"
Hermes: 스크린샷에 보이는 에러는 `ENOSPC: no space left on device`입니다.
        디스크 공간이 부족합니다. 다음 명령으로 큰 파일을 찾을 수 있습니다.
        du -sh /var/log/* | sort -rh | head -10
        실행할까요?
```

Private Chat Topics

Telegram Bot API 9.4(2026년 2월)부터 봇이 1:1 DM 안에서 **포럼 스타일 토픽**을 만들 수 있습니다. 슈퍼그룹 없이도 하나의 DM에서 여러 작업 공간을 분리할 수 있습니다.

활용 예시

장기 프로젝트를 여러 개 진행할 때 토픽별로 컨텍스트를 격리합니다.

- **"웹사이트"** 토픽 — 운영 중인 웹 서비스 관련 작업
- **"리서치"** 토픽 — 논문 조사, 자료 정리
- **"일반"** 토픽 — 잡다한 질문

각 토픽은 독립된 세션·대화 이력·컨텍스트 창을 가집니다.

설정

`config.yaml`에 토픽을 정의합니다.

```
platforms:
  telegram:
    extra:
      dm_topics:
      - chat_id: 123456789        # Telegram 사용자 ID
        topics:
        - name: General
          icon_color: 7322096
        - name: Website
          icon_color: 9367192
        - name: Research
          icon_color: 16766590
          skill: arxiv             # 이 토픽에서 자동으로 로드할 스킬
```

필드	필수	설명
`name`	예	토픽 표시 이름
`icon_color`	아니오	Telegram 아이콘 색상 코드 (정수)
`icon_custom_emoji_id`	아니오	커스텀 이모지 ID
`skill`	아니오	새 세션 시작 시 자동 로드할 스킬
`thread_id`	아니오	토픽 생성 후 자동 기록됨 — 직접 설정하지 마세요

작동 방식

1. 게이트웨이 시작 시 `thread_id`가 없는 토픽에 대해 `createForumTopic` API를 호출합니다
2. 생성된 `thread_id`는 `config.yaml`에 자동 저장되어 재시작 시 API를 다시 호출하지 않습니다
3. 각 토픽은 `telegram:dm:{chat_id}:{thread_id}` 형태의 독립 세션 키를 가집니다

`skill` 필드가 지정된 토픽은 세션이 시작될 때 해당 스킬을 자동으로 로드합니다. `/스킬이름`을 직접 입력한 것과 같은 효과입니다.

실전 예시: 멀티스텝 작업

Telegram을 통한 복잡한 작업의 실제 대화 흐름입니다.

사용자: 프로젝트 A의 최신 로그에서 에러만 추출해서 요약해줘

Hermes: 로그 파일을 확인하겠습니다.
🔍 /var/log/project-a/app.log 분석 중...

최근 24시간 에러 요약:
1. ConnectionTimeout (23건) - Redis 연결 시간 초과
→ 주로 02:00~03:00에 집중 발생
2. OutOfMemoryError (3건) - 힙 메모리 부족
→ 배치 작업 실행 시 발생
3. FileNotFoundError (1건) - /tmp/cache.dat 누락

가장 시급한 것은 Redis 연결 문제입니다. 조치할까요?

사용자: Redis 설정 확인해봐

Hermes: Redis 설정을 확인했습니다.
- timeout: 0 (무제한 대기)
- maxclients: 128
- 현재 연결 수: 127/128 ← 거의 가득 찼습니다

maxclients를 256으로 늘리면 해결될 것 같습니다.
redis.conf를 수정하고 재시작할까요?

사용자: 응 해줘

Hermes: ✅ 완료했습니다.
- maxclients: 128 → 256 변경
- Redis 재시작 완료
- 현재 연결 수: 127/256 (정상)

09-3. 디스코드 연동

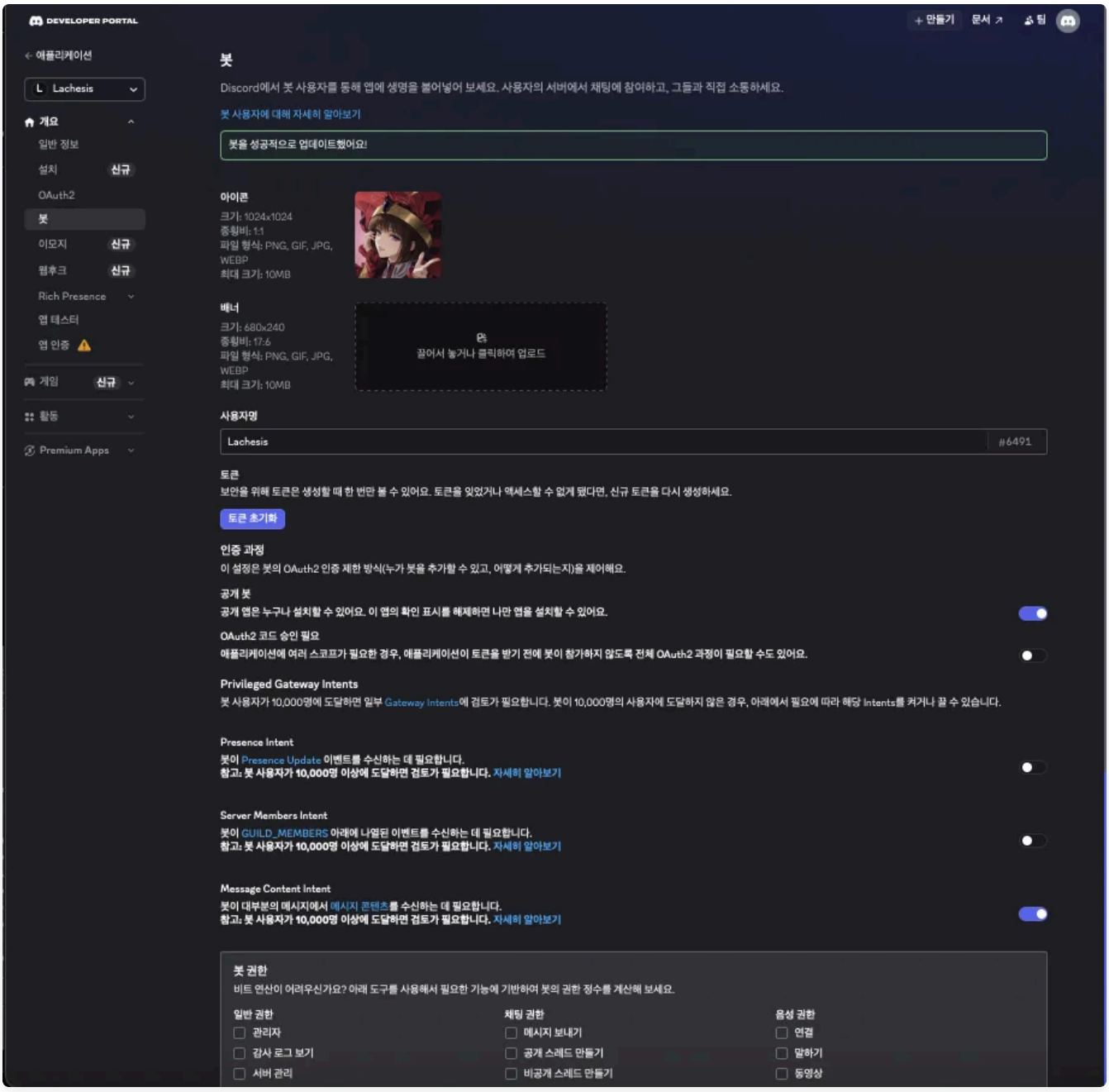
> ⚠️ ***부분 검증**: config.yaml 키·인텐트 요구사항은 v0.17.0 소스로 확인했고, 연결 흐름은 2026-06-23 omen45(Ubuntu 24.04)에서 실제로 연결해 확인했습니다. 팀·슬래시 명령 등 나머지 라이브 동작은 검증하지 않았습니다.*

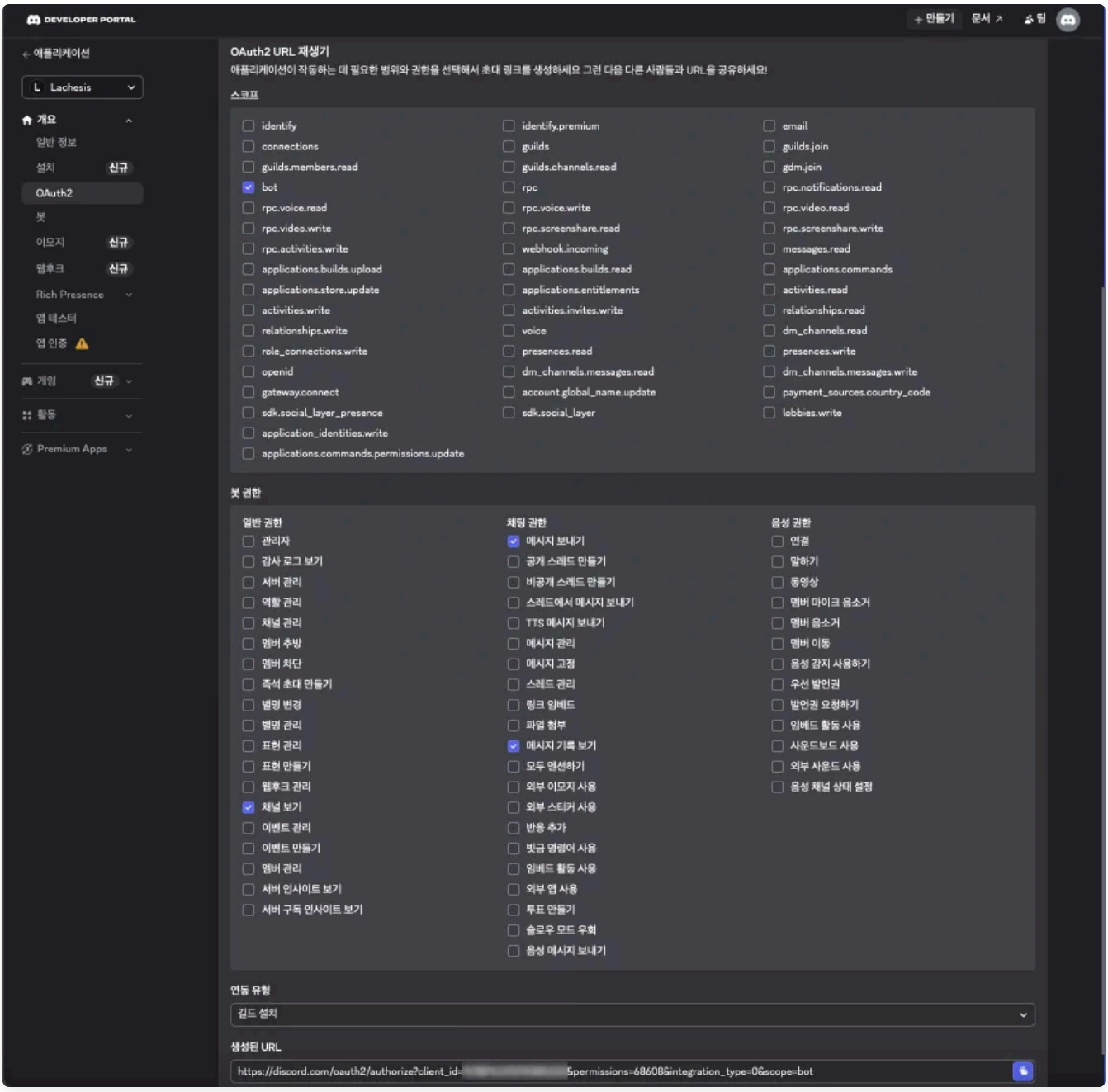
- 봇 생성
- 게이트웨이에 연결
  - 직접 편집으로 설정하기
- 사용
- 팀 운영
  - 그룹 채팅 — 세션 분리 vs 공유
  - 스레드 기반 세션
  - 채널 구조 예시
  - 팀 운영 설정
  - 권한 분리
- 고급 옵션
  - 옵트인 도구 셋
  - 슬래시 명령 자동완성과 32자 한계
- Discord vs Telegram 비교

Discord 봇으로 Hermes Agent를 연동하는 방법입니다.

봇 생성

1. Discord Developer Portal에 접속해 **New Application**으로 애플리케이션을 만듭니다.
2. **Bot** 탭에서 봇 토큰을 생성·복사합니다. 이 토큰은 아래 「게이트웨이에 연결」 단계에서 입력하므로 잠시 보관해 둡니다.
3. 같은 **Bot** 탭의 **Privileged Gateway Intents**에서 **Message Content Intent**를 켜고, 하단의 **Save Changes**까지 누릅니다. 안 켜면 봇이 메시지 내용을 읽지 못하고(멘션은 감지해도 내용은 모름), 게이트웨이가 `PrivilegedIntentsRequired` 오류로 접속하지 못합니다. 허용 목록을 숫자 ID로만 쓰면 Server Members Intent는 필요 없고, 사용자명·역할로 지정할 때만 켭니다.
4. 봇을 들일 **서버**를 준비합니다. Discord에서는 채널이든 DM이든 봇과 대화하려면 봇이 들어간 서버가 하나는 있어야 합니다. 서버가 없으면 왼쪽 막대의 **+** → **직접 만들기**로 개인 서버를 먼저 만듭니다.
5. **OAuth2** → **URL Generator**에서 `bot` 스코프와 메시지 권한을 골라 초대 URL을 만든 뒤, 그 서버로 봇을 초대합니다.

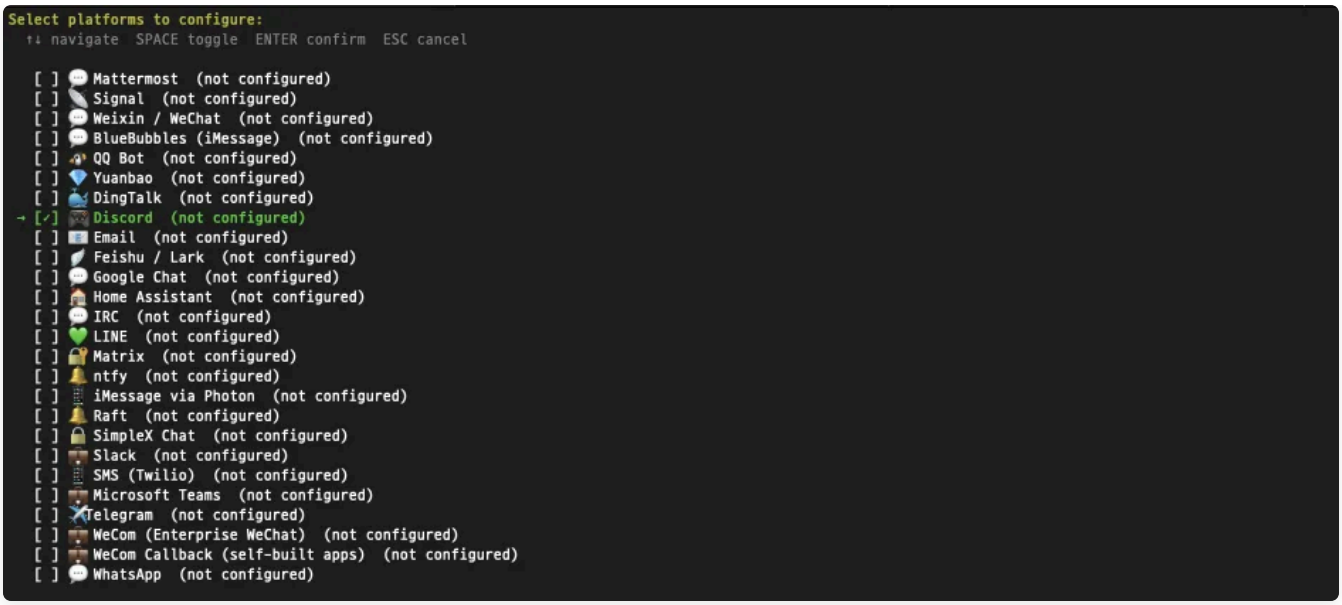

DEVELOPER PORTAL
+ 만들기
문서
팀
애플리케이션
Lachesis
개요
일반 정보
설치
신규
OAuth2
봇
이모지
신규
웹후크
신규
Rich Presence
앱 테스터
앱 인증
게임
신규
활동
Premium Apps
봇
Discord에서 봇 사용자를 통해 앱에 생명을 불어넣어 보세요. 사용자의 서버에서 채팅에 참여하고, 그들과 직접 소통하세요.
봇 사용자에 대해 자세히 알아보기
봇을 성공적으로 업데이트했어요!
아이콘
크기: 1024x1024
종횡비: 1:1
파일 형식: PNG, GIF, JPG, WEBP
최대 크기: 10MB
배너
크기: 680x240
종횡비: 17:6
파일 형식: PNG, GIF, JPG, WEBP
최대 크기: 10MB
끌어서 놓거나 클릭하여 업로드
사용자명
Lachesis
#6491
토큰
보안을 위해 토큰은 생성할 때 한 번만 볼 수 있어요. 토큰을 잊었거나 액세스할 수 없게 됐다면, 신규 토큰을 다시 생성하세요.
토큰 초기화
인증 과정
이 설정은 봇의 OAuth2 인증 제한 방식(누가 봇을 추가할 수 있고, 어떻게 추가되는지)을 제어해요.
공개 봇
공개 앱은 누구나 설치할 수 있어요. 이 앱의 확인 표시를 해제하면 나만 앱을 설치할 수 있어요.
OAuth2 코드 승인 필요
애플리케이션에 여러 스코프가 필요한 경우, 애플리케이션이 토큰을 받기 전에 봇이 참가하지 않도록 전체 OAuth2 과정이 필요할 수도 있어요.
Privileged Gateway Intents
봇 사용자가 10,000명에 도달하면 일부 Gateway Intents에 검토가 필요합니다. 봇이 10,000명의 사용자에 도달하지 않은 경우, 아래에서 필요에 따라 해당 Intents를 켜거나 끌 수 있습니다.
Presence Intent
봇이 Presence Update 이벤트를 수신하는 데 필요합니다.
참고: 봇 사용자가 10,000명 이상에 도달하면 검토가 필요합니다. 자세히 알아보기
Server Members Intent
봇이 GUILD_MEMBERS 아래에 나열된 이벤트를 수신하는 데 필요합니다.
참고: 봇 사용자가 10,000명 이상에 도달하면 검토가 필요합니다. 자세히 알아보기
Message Content Intent
봇이 대부분의 메시지에서 메시지 콘텐츠를 수신하는 데 필요합니다.
참고: 봇 사용자가 10,000명 이상에 도달하면 검토가 필요합니다. 자세히 알아보기
봇 권한
비트 연산이 어려우신가요? 아래 도구를 사용해서 필요한 기능에 기반하여 봇의 권한 정수를 계산해 보세요.
일반 권한
관리자
감사 로그 보기
서버 관리
채팅 권한
메시지 보내기
공개 스레드 만들기
비공개 스레드 만들기
음성 권한
연결
말하기
동영상



게이트웨이에 연결

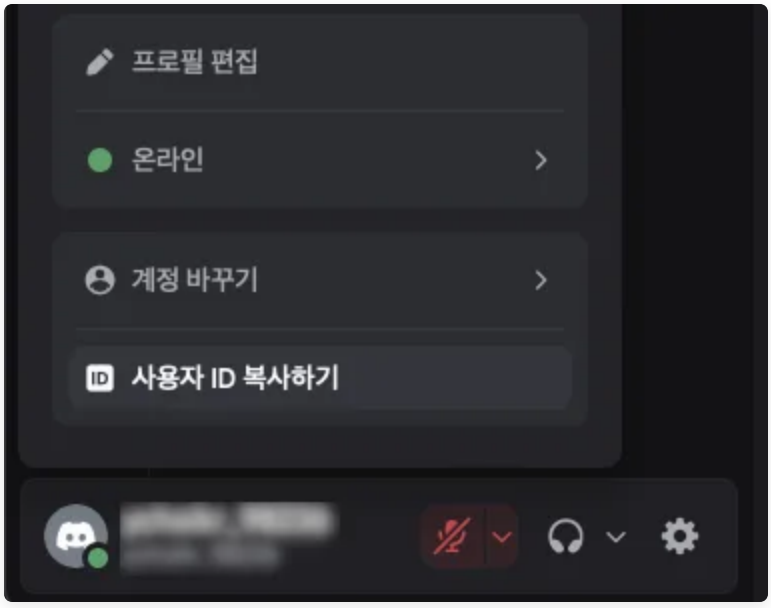
가장 간단한 길은 대화형 마법사입니다.

```
hermes setup gateway
```



Discord를 고른 뒤 차례로 입력합니다.

1. **봇 토큰** — Bot 탭에서 받은 토큰입니다(General Information의 Application ID·Public Key가 아닙니다). 토큰은 비밀이라 노출되면 Reset Token으로 무효화합니다.
2. **허용 사용자** — 본인 Discord 사용자 ID(숫자, 개발자 모드 → 우클릭 → ID 복사). 기본이 "전부 차단"이라 비우면 open access가 되어 누구나 봇을 조종할 수 있습니다. 터미널 접근 봇이므로 반드시 본인 ID를 넣습니다.
3. **홈 채널**(선택) — 크론 결과·크로스플랫폼 메시지를 보낼 채널. 비워 두고 나중에 원하는 채널에서 `/set-home` 으로 정해도 됩니다.



마법사 끝에 "Restart the gateway to pick up changes?"가 나오면 **Y**를 누릅니다. 서비스로 설치돼 있으면(9-1절) 이 재시작으로 설정이 반영됩니다.

직접 편집으로 설정하기

마법사 대신 설정 파일을 직접 편집해도 됩니다. 시크릿과 허용 목록 두 곳에 나눠 들어갑니다.

시크릿 — `~/.hermes/.env`

```
DISCORD_BOT_TOKEN=발급받은_봇_토큰
```

허용 목록 — `~/.hermes/config.yaml` (또는 `.env`의 `DISCORD_ALLOWED_USERS`)

```
discord:
  allow_from:
    - "본인_discord_user_id"
```

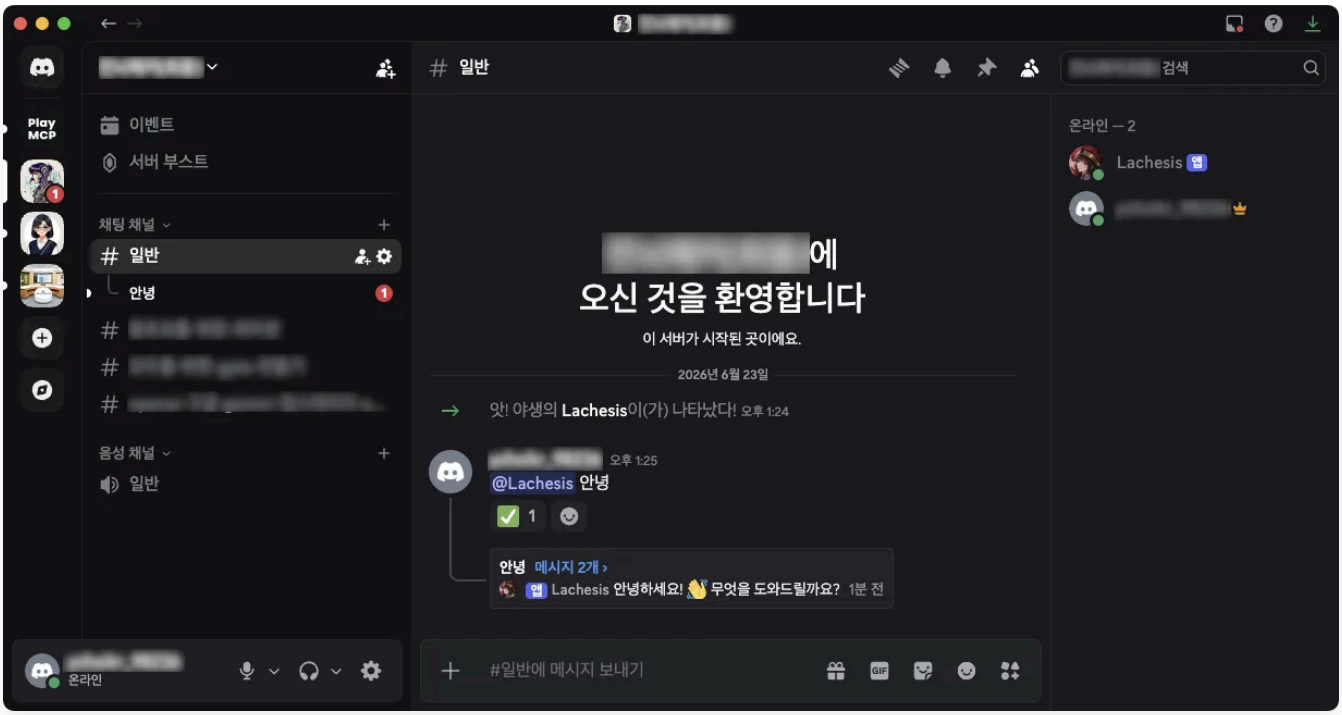
`allow_from`은 기본값이 "전부 차단"이라 비워 두면 모든 요청이 거부됩니다. 직접 고친 뒤에는 `hermes gateway restart`로 반영합니다.

연결 상태를 확인합니다.

```
hermes gateway status                    # active (running)인지
journalctl --user -u hermes-gateway -f   # Discord 로그인·메시지 처리 로그 (Linux)
```

`PrivilegedIntentsRequired` 오류가 보이면 Developer Portal에서 Message Content Intent를 켜고 **저장(Save Changes)** 했는지 확인한 뒤 다시 재시작합니다. macOS에서는 로그가 `~/.hermes/logs/gateway.log`에 쌓입니다.

봇이 온라인으로 뜨면, 초대한 서버의 텍스트 채널에서 봇을 멘션하거나 봇에게 DM을 보내 대화를 시작하면 됩니다.



채널 멘션 응답 불안정과 우회 방법

v0.17.0에서는 채널에서 봇을 멘션할 때 첫 멘션만 응답하고, 같은 채널의 이후 멘션은 응답 없이 무시됩니다. Hermes 디스코드 어댑터의 알려진 문제로 보이며, 업스트림 GitHub에 #8487로 보고돼 있습니다. 이후 대화는 자동 생성된 스레드에서 이어 가거나 봇에게 DM을 보내 진행합니다.

사용

- 봇을 멘션하거나 DM으로 대화를 시작합니다
- 슬래시 명령어(`/new` , `/model` 등)를 사용할 수 있습니다
- 에이전트가 선택지를 물으면 클릭 가능한 버튼으로 표시됩니다
- 파일 첨부 → 파일 분석
- 이미지 첨부 → 비전 분석

팀 운영

Discord의 강점은 여러 사용자가 하나의 Hermes Agent 봇을 공유하는 팀 환경입니다. 세션을 어떻게 분리하고 어떤 채널 구조로 운영할지를 모았습니다.

그룹 채팅 — 세션 분리 vs 공유

```
group_sessions_per_user: true   # true: 사용자별 세션(기본값), false: 공유 세션
```

그룹 채널에서 `true`(기본값)이면 각 사용자가 독립된 세션을 가지고, `false`이면 채널의 모든 사용자가 하나의 세션을 공유합니다.

스레드 기반 세션

Discord 스레드에서 봇을 멘션하면 해당 스레드 내에서 독립된 대화가 유지됩니다. 이를 활용하면 하나의 채널에서 여러 주제의 작업을 동시에 정리할 수 있습니다.

- 메인 채널에서 봇을 멘션하면 기본 세션으로 응답
- 스레드 안에서 멘션하면 스레드별 세션으로 응답
- 스레드 제목으로 작업 주제를 명확히 구분 가능

스레드는 Discord에서 Hermes Agent를 사용하는 가장 깔끔한 방법입니다. 주제별로 대화를 분리하면서도 하나의 채널에서 모든 것을 관리할 수 있습니다.

채널 구조 예시

```
#hermes-general     — 공유 세션, 팀 공통 질문
#hermes-devops      — 배포/인프라 관련 작업 전용
#hermes-private     — 관리자 전용 (allow_from 제한)
```

팀 운영 설정

```
# config.yaml
discord:
  allow_from:
    - "팀원1_id"
    - "팀원2_id"
    - "팀원3_id"

group_sessions_per_user: false  # 팀 채널은 공유 세션 권장
```

공유 세션(`false`)을 사용하면 한 팀원이 시작한 작업의 맥락을 다른 팀원이 이어받을 수 있습니다. 예를 들어, 팀원 A가 로그 분석을 요청한 뒤 팀원 B가 추가 조치를 지시하는 흐름이 가능합니다.

권한 분리

채널별로 다른 수준의 접근을 설정하려면 Discord의 역할(Role) 시스템과 `allow_from`를 조합합니다.

- **일반 채널** — 모든 팀원이 읽기·쓰기 가능, 위험 명령은 에이전트의 승인 모드(`approvals.mode: manual`)로 차단
- **관리자 채널** — 특정 사용자만 접근, 자동 실행 모드(`approvals.mode: off`) 허용

고급 옵션

옵트인 도구 셋

Discord 도구 셋은 `discord` **(메시지 조회·답글)와** `discord_admin` **(채널/역할 관리) 둘 다 기본 비활성**입니다. 쓰려면 최상위 `platform_toolsets` 에서 플랫폼별로 명시해 켭니다.

```
platform_toolsets:
  discord:
    - discord            # 메시지 조회·답글 (기본 비활성)
    - discord_admin      # 채널/역할 관리 (기본 비활성)
```

슬래시 명령 자동완성과 32자 한계

활성 스킬은 Telegram과 마찬가지로 Discord 슬래시 명령으로 등록되며, **외부 디렉터리(** `external_dirs` **)에 둔 스킬도 자동완성에 포함**됩니다(`/reload-skills` 후 즉시 반영). Discord 명령 이름은 32자가 한계인데, 같은 32자 클램핑 규칙을 거쳐 두 스킬이 같은 이름이 되면 게이트웨이가 경고를 남깁니다 — 이런 경우 한쪽 스킬 이름을 짧게 다듬어야 합니다.

Discord vs Telegram 비교

항목	Telegram	Discord
최적 사용	개인, 1:1 사용	팀, 다수 사용자
설정 난이도	BotFather로 2분	Developer Portal에서 5분
그룹 기능	기본적	채널/스레드/역할 등 풍부
음성 메시지	기본 지원	지원
파일 크기 제한	50MB (봇 API)	25MB (부스트·Nitro 시 최대 500MB)
봇 멘션	불필요 (DM)	멘션 또는 DM 필요
알림 커스터마이징	제한적	채널/역할별 세밀한 제어

간단 정리:

- 혼자 쓸 때는 **Telegram**이 설정도 쉽고 DM으로 바로 대화할 수 있어 편리합니다.
- 팀에서 공유할 때는 **Discord**가 채널 분리, 역할 관리, 스레드 등의 기능으로 더 적합합니다.

- 두 플랫폼을 동시에 운영하는 것도 가능합니다. 관리자는 Telegram DM, 팀은 Discord 채널을 사용하는 조합이 실용적입니다.

09-4. Microsoft Teams 연동

> ⚠️ ***부분 검증**: 이 페이지는 v0.16.0 소스(`microsoft-teams-apps` SDK 기반 Bot Framework 어댑터 `plugins/platforms/teams/adapter.py`)와 공식 문서(`website/docs/user-guide/messaging/teams.md`)를 대조해 작성했습니다. 봇 등록·실제 메시지 송수신은 외부 의존성(Microsoft Teams 앱 등록, 공용 HTTPS 엔드포인트)이 필요해 직접 운영 테스트는 하지 않았습니다. 환경에 따라 절차가 달라질 수 있습니다.*

- 메시지 파이프라인
- 할 수 있는 일
- 사전 준비
- 설정
- 게이트웨이 시작
- 메시지 보내기
- 한계와 주의 사항
- 다른 플랫폼과의 차이

Microsoft Teams 봇으로 Hermes Agent를 연결하는 방법입니다. Hermes의 Teams 연동은 `microsoft-teams-apps` SDK를 쓰는 **Bot Framework 봇**으로, 봇의 메시지 엔드포인트(`/api/messages` , 기본 포트 3978)로 들어오는 메시지를 받아 처리합니다.

메시지 파이프라인

1. **인증** — Azure AD 앱의 클라이언트 시크릿으로 **Bot Framework 토큰**을 발급받습니다(SDK가 처리).
2. **메시지 수신** — Teams Bot Framework가 봇의 메시지 엔드포인트(`/api/messages` , 기본 포트 3978)로 메시지를 전달하고, 어댑터의 HTTP 서버가 이를 받습니다. 엔드포인트는 Bot Framework JWT로 인증되며, 별도의 Microsoft Graph 구독 갱신은 필요 없습니다.
3. **필터링** — 수신 메시지에서 사용자 식별자(AAD object ID)를 뽑아 `TEAMS_ALLOWED_USERS` 로 거른 뒤, 통과한 메시지를 에이전트 세션에 전달합니다.
4. **송신** — 에이전트의 응답을 Bot Framework를 통해 같은 대화(채널·그룹·DM)로 보냅니다.

할 수 있는 일

- Teams **채널·그룹·1:1 DM** 어디에서나 에이전트와 대화할 수 있습니다(DM은 멘션 없이, 그룹·채널은 봇을 멘션해야 응답).

- 위험 명령 승인을 **Adaptive Card** 버튼으로 받을 수 있습니다.
- 크론 작업 결과를 지정한 채널(홈 채널)로 보낼 수 있습니다.
- 별도의 **Teams 회의 요약 파이프라인**(Microsoft Graph 기반)도 제공됩니다.
- Telegram, Discord 같은 다른 메시징 플랫폼과 함께 운영할 수 있습니다.

사전 준비

봇 등록은 **Teams CLI**로 합니다.

1. Teams CLI를 설치합니다.

```
npm install -g @microsoft/teams.cli@preview
teams login
```

2. 게이트웨이가 들을 포트(기본 3978)를 공용 HTTPS로 노출합니다. 로컬 개발에서는 devtunnel·ngrok·cloudflared 같은 터널을 씁니다.

3. 봇을 생성하면서 메시지 엔드포인트를 지정합니다.

```
teams app create --name "Hermes" --endpoint "https://<tunnel>/api/messages"
```

4. 출력된 `CLIENT_ID` · `CLIENT_SECRET` · `TENANT_ID` 를 저장하고, 설치 링크로 워크스페이스에 추가합니다.

```
teams app get <app-id> --install-link
```

설정

`~/.hermes/.env` 에 인증 정보와 허용 사용자를 저장합니다.

```
TEAMS_CLIENT_ID=<client-id>
TEAMS_CLIENT_SECRET=<client-secret>
TEAMS_TENANT_ID=<tenant-id>
# 접근을 허용할 사용자(AAD object ID). teams status --verbose로 확인
TEAMS_ALLOWED_USERS=<your-aad-object-id>
```

`config.yaml` 에서 플랫폼을 켜고 포트 등 세부를 지정할 수 있습니다.

```
platforms:
  teams:
    enabled: true
    extra:
      client_id: "your-client-id"
      client_secret: "your-secret"
      tenant_id: "your-tenant-id"
      port: 3978
```

기본값은 전체 차단입니다

`TEAMS_ALLOWED_USERS`를 설정하지 않고 `TEAMS_ALLOW_ALL_USERS=true`도 주지 않으면, 봇은 모든 요청을 거부합니다(기본 차단). 승인 카드 클릭도 같은 정책을 따릅니다. `*`를 넣으면 전체 허용입니다. 봇이 터미널·파일 권한을 가질 수 있다는 점을 고려해 신중하게 설정합니다.

게이트웨이 시작

```
hermes gateway
```

봇 메시지를 받으려면 게이트웨이가 실행 중이어야 하고, 3978 포트(또는 `TEAMS_PORT`로 지정한 포트)가 공용 HTTPS 엔드포인트로 노출돼 있어야 합니다.

메시지 보내기

채널이나 사용자를 지정하면 에이전트가 직접 메시지를 보낼 수 있습니다. 크론 작업 결과를 받을 채널을 설정하는 방법은 9장 도입부의 `/sethome` 안내를 참고합니다.

한계와 주의 사항

- 봇 등록과 워크스페이스 설치에는 조직(테넌트) 권한이 필요할 수 있습니다. 회의 요약 파이프라인처럼 Microsoft Graph 권한을 쓰는 기능은 별도로 관리자 동의가 필요합니다.
- 봇이 받은 메시지는 Microsoft 측 로그에 남습니다. 사내 컴플라이언스 정책과 맞는지 미리 확인합니다.
- 메시지 본문은 약 28,000자까지 처리됩니다. 큰 파일은 외부 스토리지에 올린 뒤 링크로 공유합니다.

다른 플랫폼과의 차이

Telegram이나 Discord는 봇 API 구조가 단순하고 설정 절차도 짧습니다. 반면 Teams는 Bot Framework와 Teams CLI, 공용 엔드포인트를 함께 써야 하므로 초기 설정이 더 복잡합니다. 대신 한 번 설정해 두면 사내 업무 흐름에 자연스럽게 녹아듭니다. 동료들은 채널에서 봇을 멘션하는 방식으로 에이전트를 사용할 수 있습니다.